Vroomie

Audio Diagnostics Architecture & Engineering Handover

Complete technical reference for architects and development teams extending the Vroomie audio anomaly-detection system

Engine version v9.7 · ETHANOL-FEATURE
github.com/DGeorgeA/vroomie-app · branch main
Live at vroomie.in

All performance figures herein are measured results on held-out data, reproducible with the committed test harnesses.

0. How to read this document

Sections 1–4 are the system as it stands. **Section 5 is the most valuable part**: a catalogue of failures that already happened, what caused them, and the measurement that proved it. Several of these look like obvious "improvements" to a newcomer and are in fact regressions that were shipped, measured, and reverted.

Section 9 is a hard "do not do this" list. Read it before changing any constant.

---

1. Executive summary

Vroomie records engine audio through a phone microphone and identifies mechanical faults by matching against a curated reference library stored in Supabase Storage (bucket `anomaly-patterns`).

Current measured performance (held-out data, not training data):

Metric	Value
Bucket reference files detected (played at mic)	**54 / 55**
Healthy-vehicle false positives	**0 / 35**
Interferer false positives (speech, TV, music, noise, fan, tone)	**0 / 9**
Held-out real-world fault recall	**34 / 47 (72%)**
Long-session (60 s) healthy false positives	**0 / 22**
Intermittent playback (tap-play testing)	**12 / 12**

The single non-detected file is `water_pump_failure_critical.wav`, which is a synthetic sine tone, not a recording. It is explicitly excluded by design (see §5.9).

Core architectural principle: detection asks *"is this closer to a known fault than to a healthy engine?"* — never *"which fault is closest?"*. That distinction is the entire reason the system stopped hallucinating. See §3.3.

---

2. System topology

```
Phone microphone (getUserMedia, AGC/NS/EC disabled)
  ↓ native-rate AudioContext  ← NOT forced 16 kHz (iOS bug, §5.4)
  ↓ ScriptProcessor, 1-second windows every ~0.9 s
  ↓ software resample → 16 kHz
  ↓ RMS gate (raw < 0.005 → reject as silence)
  ↓ RMS normalize to 0.05      ← MUST match reference factory (§5.3)
  ↓
YAMNet (TF.js, from TF Hub, cached by service worker)
  ├── 1024-d mean embedding      → used for matching
  └── 521-class mean scores      → used for the domain gate
  ↓
STAGE 1  Acoustic domain gate     — is this vehicle audio at all?
STAGE 2  Discriminative match     — closer to a fault than to healthy?
STAGE 3  Session fraction rule    — sustained across the session?
  ↓
Anomaly result  (label, fault_type family, possibility %, source file)
  ↓
[OPTIONAL] Ethanol interpretation layer — pure consumer, zero influence
```

```
  ↓
Supabase `analyses` table + report UI + PDF
```

2.1 Key files

File	Role
`src/lib/audioFeatureExtractor.js`	Mic capture, windowing, resample, normalize, calls engine
`src/lib/mlEmbeddingEngine.js`	YAMNet load, domain gate, margin rule, confidence
`src/lib/datasetLoader.js`	Loads the static reference artifact (no in-browser generation)
`public/fingerprints_v9.json`	**The reference artifact** — 560 fault embeddings + 93 anchors
`scripts/build_reference_fingerprints.mjs`	**Offline reference factory** — regenerates the artifact
`src/components/predictive/AudioRecorder.jsx`	Session state, fraction rule, report payload
`src/lib/ethanolScreening.js`	Ethanol interpretation (pure function)
`src/lib/motionDetector.js`	Accelerometer annotation (not a gate — §5.6)
`ethanol_feature_setup.sql`	RBAC + feature flag schema with RLS

---

3. The detection engine in detail

3.1 Stage 1 — Acoustic domain gate

YAMNet's 521-class scores decide whether a window is vehicle/mechanical audio *before* any fingerprint comparison.

- **Accept** if top-1 class is in `VEHICLE_MECH_NAMES` (53 classes: Engine, Idling, Rattle, Gears, …)
- **Accept** if top-1 is a *generic* acoustic class (White noise, Sine wave, Explosion…) **and** interferer evidence < 0.15
- **Accept** if strongest vehicle class ≥ 0.03 **and** > strongest interferer class
- **Reject** otherwise

`INTERFERER_INDICES` = all human-sound classes + all music classes + Television, Radio, Silence, Whistling, Whistle (216 classes).

Deliberately excluded from the accept list: `Mechanical fan`, `Air conditioning`. Household fans passed the gate and matched hiss-like references.

Why the "generic acoustics" clause exists: real faults frequently do *not* classify as vehicle sounds. Measured: alternator bearing → "White noise"/"Waterfall"; misfire → "Explosion"/"Rain"; pump whine → "Sine wave". A strict vehicle-only gate dropped those windows entirely.

3.2 Stage 2 — Discriminative match (the margin rule)

```
bestFault  = max cosine similarity vs 560 fault embeddings
bestAnchor = max cosine similarity vs 93 anchors (healthy engines + interferers)
margin     = bestFault − bestAnchor

candidate     bestFault ≥ 0.45  AND  margin ≥ 0.04
```

The anchors are the critical invention. Without them, "find the closest fault" is meaningless because YAMNet embeddings of *any* two sustained sounds sit at 0.7–0.9 cosine. Measured evidence: a healthy idling car scores **0.892–0.918** against the power-steering fault references — higher than several genuine faults. The anchors supply the "compared to what?".

3.3 Stage 3 — Session fraction rule (in `AudioRecorder.jsx`)

```
confirmed   ≥45% of gate-accepted windows are candidates for the SAME
             fault FAMILY, with ≥3 accepted windows minimum
```

Votes aggregate by `fault_type` **family**, not by individual label. Reason: three separate alternator-bearing references split the vote three ways, so a session with 57% total bearing candidates confirmed nothing. The dominant label within the winning family is what gets reported.

3.4 Operating point — and how it was chosen

Constant	Value	Location
`ANOMALY_THRESHOLD`	0.45	`mlEmbeddingEngine.js`
`ANCHOR_MARGIN`	0.04	`mlEmbeddingEngine.js`
`SESSION_FRACTION`	0.45	`AudioRecorder.jsx`
`SESSION_MIN_ACCEPTED`	3	`AudioRecorder.jsx`
RMS silence gate	0.005 (raw)	`audioFeatureExtractor.js`
Normalization target	0.05 RMS	both live + factory
Domain gate interferer ceiling	0.15	`mlEmbeddingEngine.js`
Vehicle score floor	0.03	`mlEmbeddingEngine.js`

These were **selected by grid search**, not intuition: maximise recall subject to *zero* healthy-vehicle and *zero* interferer false positives. The QA suite asserts them, so a change fails the build.

3.5 Confidence ("possibility %")

```
possibility = clamp(70% + 108 × (margin − 0.04), 70%, 97%)
```

Derived from the **margin**, i.e. the match strength after subtracting the best healthy/noise anchor — "discounting the noise". Raw cosine is *not* used, because it sits at 0.7–0.9 for any two sustained sounds and previously produced meaningless "80% confidence" claims. An anomaly cannot be confirmed below the margin threshold, so every published statement is ≥70% by construction.

Reported as: *"There is a 92% possibility that there could be a possible PowerSteeringPump (PowerSteeringPump.wav)"*

---

4. The reference factory (offline)

References are never generated in the browser. `scripts/build_reference_fingerprints.mjs` produces `public/fingerprints_v9.json`, which is committed and service-worker precached.

Pipeline per reference file:

1. **QC** — reject synthetic tones (YAMNet dominant class □ {Sine wave, Harmonic, Chirp tone, …}), files < 1 s, near-silent files, plus an explicit `EXCLUDED_REFERENCES` list
2. **Preprocess** — mono, 16 kHz, RMS-normalize to 0.05
3. **Augment** — 6 variants per chunk: original, phone-band filter, +15 dB-SNR noise, ±2% rate shift, room echo, **speaker-replay** (300–8000 Hz bandpass + dual echo)
4. **Embed** — YAMNet, int8-quantized

Anchors (the "compared to what?"): healthy idle/startup/brakes from the Kaggle dataset (EVEN-numbered clips only — odd are held out for evaluation) + speech, music, white noise ×3 levels, pink noise, fan simulation, pure tone.

4.1 Adding a new fault class — the supported workflow

1. Upload a **real recording** (not synthetic) to the `anomaly-patterns` bucket, named descriptively (the filename becomes the label and drives `fault_type` via `deriveMeta`)
2. Alternatively drop it in `reference_audio/` for curated local references (dense 8-chunk coverage)
3. Run `node scripts/build_reference_fingerprints.mjs`
4. Run `node scripts/audit_all_bucket_files.mjs` and the QA suites
5. Commit the regenerated `fingerprints_v9.json`

Do not hand-edit the artifact. **Do not** add in-browser fingerprint generation (removed for good reason — §5.2).

---

5. Failure catalogue — read this before changing anything

Each entry is a real defect that shipped, was measured, and was fixed. Several were "obvious improvements" that made things worse.

5.1 The original hallucination: similarity-only matching

Symptom: silence, TV, ambient noise reported as "Power Steering Combined No Oil Serpentine Belt". **Root cause (measured):** the engine matched on cosine ≥ 0.75 with no check that audio was vehicular. Ambient room noise scores **0.86–0.89** against hiss-like references. Additionally, the reference set was **74–81% one class**, so a nearest-neighbour matcher drifted there whenever uncertain. **Fix:** the three-stage pipeline in §3. YAMNet's class scores were already being computed and **thrown away**.

5.2 In-browser fingerprint generation

Problem: each client downloaded ~11 MB and ran YAMNet over 55 files before recording could start. Minutes of latency, localStorage quota issues, non-deterministic references. **Fix:** offline factory + static artifact. **Never reintroduce this.**

5.3 Level asymmetry — the quiet-capture killer

Symptom: "Unable to detect vehicle audio" regardless of what was played. **Root cause:** the factory normalizes every reference to 0.05 RMS, but live mic windows entered YAMNet **raw**, and anything below 0.01 RMS was discarded as silence. With AGC deliberately disabled, a phone a metre from a source captures at **0.005–0.02 RMS** — entire sessions were thrown away. **Measured impact of fix:** quiet replay 0/5 → **5/5**; held-out recall 14/36 → 21/36. **Rule:** live windows and reference chunks must undergo *mathematically identical* preprocessing. Prove it with values, not by reading code.

5.4 Forced 16 kHz AudioContext (iOS)

`new AudioContext({sampleRate: 16000})` + `createMediaStreamSource` is a known iOS Safari failure (silence / errors when context rate ≠ hardware rate). **Capture at native rate, resample in software.**

5.5 Fake-HEALTHY reports

Two paths could publish a confident "healthy" report built from zero evidence: (a) reference artifact fails to load → `no_references` was counted as a *clean* window; (b) YAMNet never loads → zero windows analysed but a report was still written. Both now abort with specific errors. **Any new failure mode must fail loudly, never as "healthy".**

5.6 The motion gate that suppressed bench tests

A vehicle-vibration gate was added, then found to **hard-suppress anomalies on a stationary phone** — exactly the posture of a controlled bench test (sample played from a speaker at a phone on a table). **Fix:** stillness now *annotates* the report ("vehicle vibration was not sensed; verify at the running vehicle") instead of suppressing it. Never hard-gate anomalies on motion.

5.7 The loosening regression (20% false positives)

A parallel workstream, chasing the same "not detecting" complaint, set τ 0.60→0.40, session fraction 0.45→0.10, min windows 4→1 (anomaly on a **single 900 ms window**). **Measured:** 20% false positives on healthy vehicles (7/35) to buy +9pp recall. **Resolution:** grid search found τ=0.45 / margin=0.04 / 45% of ≥3 windows — **better recall than the strict config AND zero false positives**. Loosening is not the answer; better *discrimination* is.

5.8 Report-layer defects that masqueraded as detection failures

Detection was measured at 54/55 while the user experienced "the app isn't working". The real causes were downstream:

- **PDF export crashed on 100% of anomaly reports** — `dict.inr.toLocaleString()` where cost fields had been purged from the dictionary. `doc.save()` never ran. *Healthy* reports exported fine, which hid it.
- Narrative lookup used **first-match**, so the 44-file combined power-steering class (the largest) was hijacked by the shorter `SerpentineBelt` key → **wrong repair advice**. Now longest-match.
- `getDiagnosticMetadata` was exact-match only → no engine label ever matched → PDFs always printed the generic fallback.
- PDF read `matchedFile`; the engine writes `sourceFile`.
- Four unguarded `.toUpperCase()`/`.toFixed()` calls crashed history/details rendering on legacy records.

Lesson: when a user reports "not detecting", verify the **report path** as well as the classifier.

5.9 Dataset contamination

- `water_pump_failure_critical.wav` is a **pure synthetic sine tone**, not a pump recording. QC rejects it and it is in `EXCLUDED_REFERENCES`. That class stays undetectable until a real recording replaces it — accepting tones would let alarm beeps and test tones flag as faults.
- The 55 `.json` files in the bucket are dead v7-era fingerprints, several with empty feature arrays and mismatched internal IDs. Nothing reads them.
- The reference set is dominated by one class (44 of 55 files are numbered variants of the same combined power-steering recording).

5.10 Cross-recording generalization is the real ceiling

Bucket references replay at 54/55, but *unseen* real-world recordings detect at 72%. A YouTube alternator-bearing recording initially failed: the gate accepted 83/83 windows, but only ~13% matched references — it sat at the edge of the reference cloud. **Fix pattern:** promote the failing recording to a curated reference (`reference_audio/`) + add speaker-replay augmentation. This is the *supported* way to improve recall.

---

6. Ethanol Contamination Check (v9.7)

An optional, admin-gated, globally flagged screening layer.

Architectural boundary (must not be crossed): `src/lib/ethanolScreening.js` **imports nothing**, holds no thresholds, and performs no audio work. It is a pure function of the anomaly result the core engine already produced. The QA suite statically asserts that no audio module references it, guaranteeing `CORE_AUDIO_RESULT(feature on) == CORE_AUDIO_RESULT(feature off)`.

- **Relevant families:** `piston_knock`, `power_steering`, `rocker_valve` — matched by canonical `fault_type` IDs, never display strings (with a legacy-label regex fallback for old records)
- **Never claims** ethanol contamination was detected; **never** gives an all-clear
- **UI:** Golden Ticket intro modal (original Vroomie design), CSS-overlay golden sash on the logo (asset untouched, so disabling leaves zero residue), result modal with visible disclaimer
- **Flow:** CHECK NOW starts the **existing** recording session via registered controls; the interpretation is applied to the finished core result

6.1 Security model

Concern	Implementation
Roles	`public.user_roles`, keyed by **Auth UUID**, never email
Admin assignment	Email resolves the UUID **once, server-side**, in SQL. No app code compares emails
New accounts	Trigger defaults every account to `user`; sole-admin enforcement demotes others
Failure behaviour	Role lookup fails **closed** to `user`; flags fail **safe** to disabled
Feature flag writes	RLS `USING (public.is_admin())` — an empty result is treated as unauthorized
Audit	`updated_by` records the acting UUID (NULL would indicate anonymous)

Live-verified: anonymous PATCH of `app_features` changes nothing; anonymous INSERT into `user_roles` → `42501 RLS violation`; anonymous `is_admin()` → false; anonymous SELECT of `user_roles` → `[]`.

---

7. Test harnesses — use these, don't write new ones

Script	Purpose	Runtime
`scripts/qa_unit_tests.mjs`	Confidence mapping, statement format, motion classifier, report guards, **operating-point assertions**	seconds
`scripts/qa_ethanol_feature.mjs`	Ethanol TEST A–H, claim safety, security invariants, audio-invariance proof	seconds
`scripts/audit_all_bucket_files.mjs`	Every bucket file through the shipped decision path, speaker-replay channel	~45 min
`scripts/benchmark_discrimination.mjs`	Held-out 91-session matrix; dumps per-window measurements	~30 min
`scripts/rule_explorer.mjs`	Instant rule/threshold grid search on the dumped measurements	seconds
`scripts/diagnose_live_replay.mjs`	Per-window telemetry: RMS, gate verdict, top-5 matches, margin, rejection reason	~20 min
`scripts/validate_intermittent.mjs`	Tap-play (clip + gap) patterns, long healthy sessions	~30 min
`scripts/build_reference_fingerprints.mjs`	Regenerate the artifact	~30 min

Workflow for any threshold change: run `benchmark_discrimination.mjs` once to dump measurements → iterate instantly with `rule_explorer.mjs` → only then edit constants → re-run the QA suites.

7.1 Browser end-to-end technique (important)

Offline harnesses replicate the pipeline's *math*. To exercise the **actual shipped modules**, inject a fake microphone:

```
navigator.mediaDevices.getUserMedia = async (c) => {
  const ctx = new AudioContext(); await ctx.resume();
  const dest = ctx.createMediaStreamDestination();
  const src = ctx.createBufferSource();
  src.buffer = decodedWav; src.loop = true;
  src.connect(dest); src.start();
  return dest.stream;
};
```

Gotchas learned the hard way:

- **Unregister the service worker first** — `autoUpdate` reloads wipe page patches
- **Wrap** `AudioContext` to track instances and resume them on any click (autoplay suspension)
- Test rows land in `analyses` with `vehicle_id = 'guest-vehicle'`; the anon key **can** delete them — clean up after testing

---

8. Operational runbook

Build markers. The sidebar shows the engine version (e.g. `v9.7-ETHANOL-FEATURE`) and every report stores `analysis_result.session_diagnostics.engine_build`. **Always confirm the marker before diagnosing a field report** — stale service workers serving old bundles have repeatedly been mistaken for detection failures.

Field diagnostics. Every report stores:

```
session_diagnostics: { windows_analyzed, accepted, rejected,
                       candidate_windows, capture_settings, engine_build }
analysis_result.motion: { verdict, vibration_rms, samples }
```

`capture_settings` records the device's **actual applied** mic constraints — some Android builds force `noiseSuppression` on, which attenuates broadband fault signatures.

Deployment. Push to `main` → GitHub Actions → GitHub Pages (`npm run build:production`). Verify the marker in the *served* bundle, not just the repo.

Enabling the Ethanol feature. Sign in as the admin account → Settings → Admin Settings · Special Features → Enable (confirms, because it is global). No rebuild or redeploy required.

---

9. Do-not-do list

1. **Do not lower thresholds to fix recall.** Measured: it produces a 20% healthy false-positive rate. Improve discrimination (references, anchors, augmentation) instead.
2. **Do not remove the healthy anchors.** They are the only thing separating "faulty engine" from "engine".
3. **Do not reintroduce in-browser fingerprint generation.**
4. **Do not force a 16 kHz AudioContext** on the mic stream.
5. **Do not break live/reference preprocessing parity** (resample + normalize to 0.05).
6. **Do not hard-gate anomalies on motion sensors.**
7. **Do not add synthetic tones as references.**
8. **Do not match on display strings** — use `fault_type` canonical IDs.
9. **Do not let any failure path publish a "healthy" result.**
10. **Do not let the ethanol (or any future optional) layer import into the audio path.**
11. **Do not trust a passing test that hardcodes constants** — a prior suite asserted a reverted margin and printed "verification complete" while verifying nothing. Tests must read constants from source.
12. **Do not claim success on a green build.** Every material claim in this system was established by measurement on held-out data.

---

10. Known limitations & highest-leverage next steps

Limitations

- Real-world recall is 72% — bounded by the reference dataset (processed 1.5 s clips, one dominant class)
- `water_pump` class disabled (synthetic reference)
- `intake_leak` replay is suppressed by the broadband-noise anchors that provide fan immunity — an intake leak *is* a hiss; this is a genuine tension, not a bug
- Injector tick and exhaust leak have **no references at all** and cannot be detected
- Devices that force noise suppression may attenuate signatures (now visible in `capture_settings`)

Highest-leverage improvements, in order

1. **Real phone-mic recordings of real vehicles** — healthy *and* faulty — uploaded to the bucket. This moves recall more than any algorithm change. The factory ingests them automatically.
2. Replace the synthetic water-pump file; add injector-tick and exhaust-leak references.

3. Per-vehicle healthy baselining (record the user's own healthy engine as a personal anchor) — the natural next architectural step, and the cleanest path past the generic-anchor ceiling.
4. Re-run the grid search after any dataset change; the current operating point is optimal *for the current dataset*, not universally.

---

11. Change-safety checklist

Before merging any change to detection:

- ■ `node scripts/qa_unit_tests.mjs` passes
- ■ `node scripts/qa_ethanol_feature.mjs` passes
- ■ `node scripts/benchmark_discrimination.mjs` → healthy FP still 0/35, interferer FP still 0/9
- ■ `node scripts/audit_all_bucket_files.mjs` → still ≥54 detected
- ■ `npm run build` clean
- ■ Browser E2E with injected audio produces a correct flagged report
- ■ Build marker bumped so field reports are attributable
- ■ `git fetch` and diff detection constants before assuming local == production

---

Prepared from the full engineering record of the v9.0 → v9.7 remediation programme. Every performance figure in this document is a measured result on held-out data, reproducible with the committed harnesses.